

Programiranje distribuiranih sistema

Mini E-commerce mikroservisna aplikacija

Izveštaj o projektu

Jovana Mitrović 80/2023

Prirodno-matematički fakultet, Univerzitet u Kragujevcu

Sadržaj

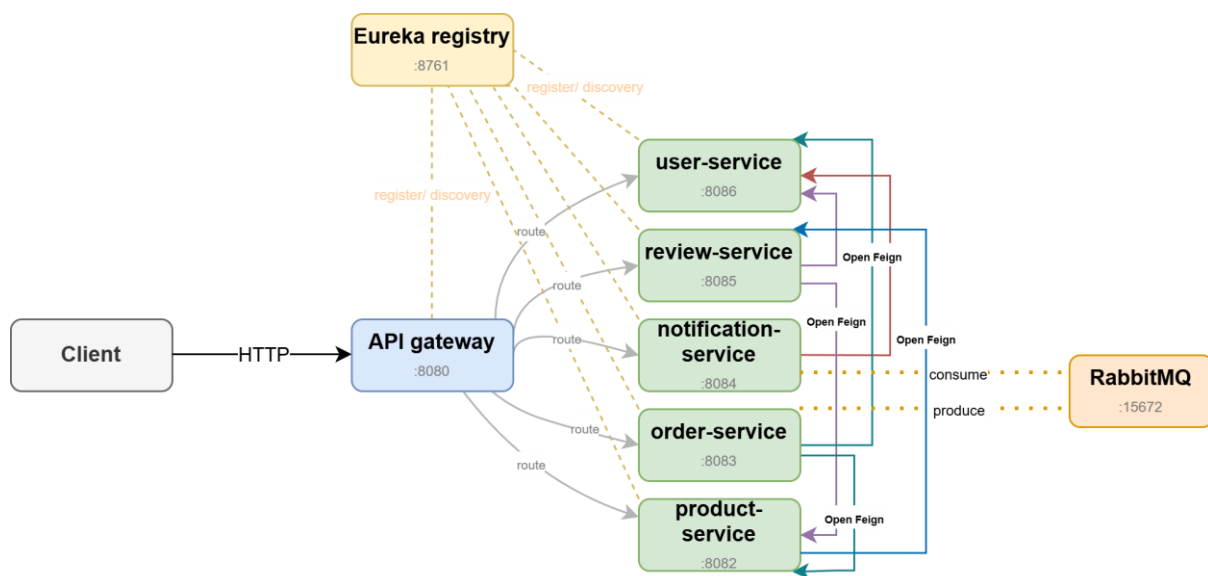
1. Opis projekta	2
2. Arhitektura sistema	3
2.1. Portovi servisa	4
3. Opis servisa	5
3.1. User Service	5
3.2. Product Service	5
3.3. Order Service	5
3.4. Review Service	6
3.5. Notification Service	6
4. Komunikacija između servisa	6
4.1. Sinhrona komunikacija — OpenFeign	6
4.1.1 Otpornost na greške — Resilience4j	7
4.3. Asinhrona komunikacija — RabbitMQ	7
5. API Gateway i Service Discovery	7
6. Rukovanje greškama	8
7. Kontejnerizacija — Docker i Docker Compose	8

1. Opis projekta

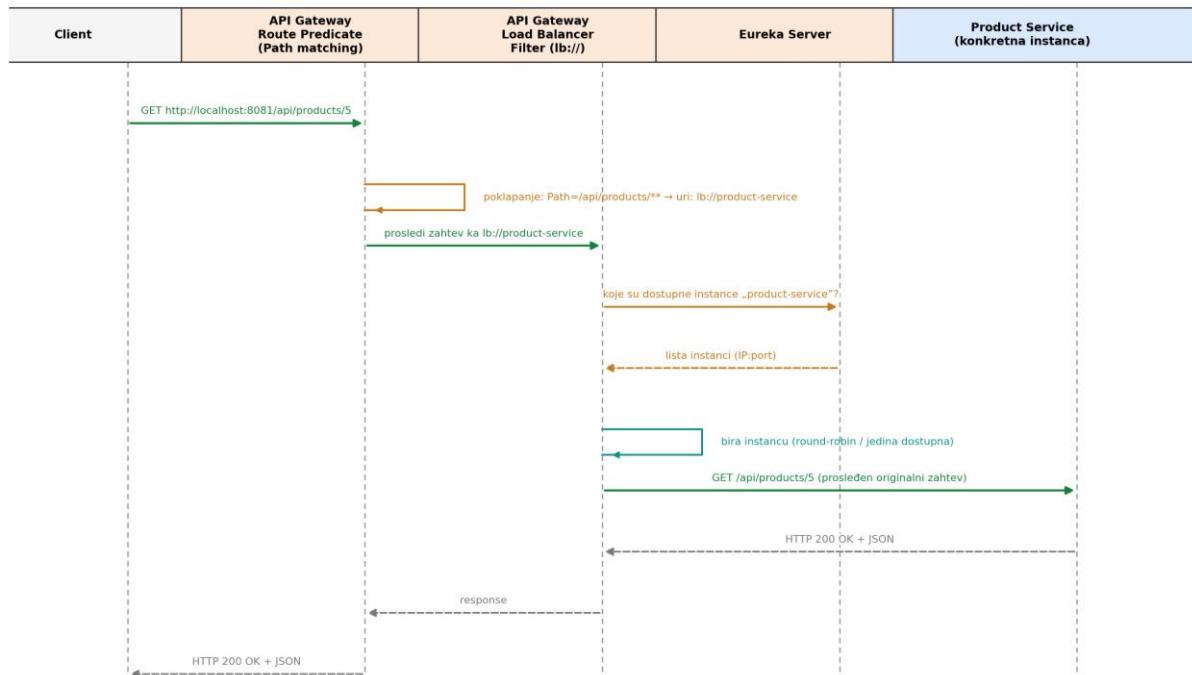
Tema projekta je Mini e-commerce mikroservisna aplikacija, sa sledećim poslovnim servisima: *user-service*, *product-service*, *order-service*, *review-service* i *notification-service*. Aplikacija demonstrira principe modernog distribuiranog sistema, u Javi sa Spring Boot-om: razdvajanje na servise, sinhronu komunikaciju preko *OpenFeign-a*, dinamičko pronalaženje servisa kroz *Eureka*, kontrolisan ulaz kroz *API Gateway*, load balancing, otpornost na greške (*Resilience4j*), asinhronu komunikaciju preko *RabbitMQ-a*, *Docker* kontejnerizaciju i globalni exception handling kroz *@RestControllerAdvice*.

2. Arhitektura sistema

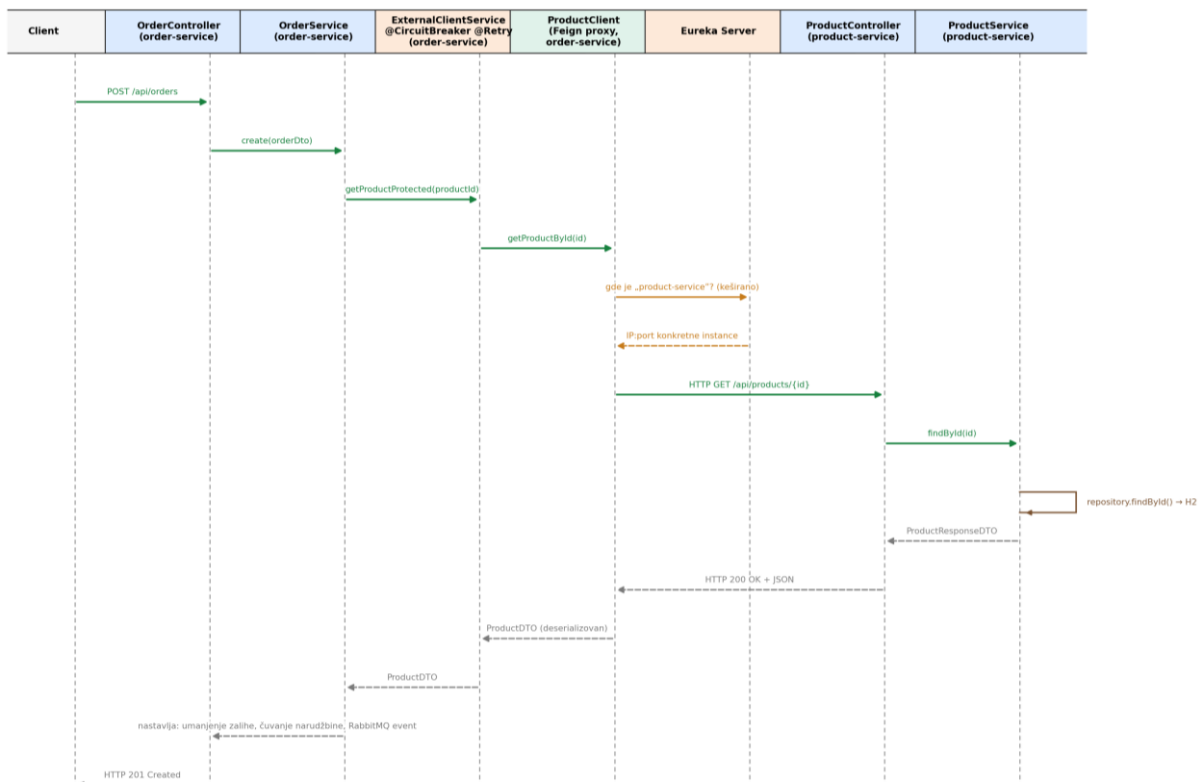
Sistem se sastoji od 7 servisa – 5 poslovnih i 2 infrastrukturna (api-gateway i eureka-discovery-server). Klijent može da komunicira sa API Gateway-om, koji na osnovu putanje rutira zahtev ka odgovarajućem servisu preko Eureka registra (load-balanced rutiranje, prefiks lb://). Svaki servis se registruje kod Eureka servera pri pokretanju. Sinhrona komunikacija između servisa realizovana je preko OpenFeign klijenata. Feign pozivi su zaštićeni Resilience4j Circuit Breaker-om i Retry mehanizmom. Asinhrona komunikacija (RabbitMQ) koristi se za kreiranje obaveštenja nakon kreiranja narudžbine, bez blokiranja glavnog toka kreiranja narudžbine.



Slika 1 — Arhitektura sistema i komunikacija između komponenti



Slika 2 — Tok zahteva kroz API gateway



Slika 3 — Tok Feign poziva (kreiranje narudžbine)

Napomena. `ProductClient` je Feign interfejs bez implementacije. Spring Cloud OpenFeign kreira JDK Dynamic Proxy koji implementira ovaj interfejs. Kada se pozove `productClient.getProductById()`, poziv "presreće" proxy, koji preko Feign infrastrukture formira HTTP zahtev.

2.1. Portovi servisa

Servis	Port	Uloga
Eureka Discovery Server	8761	Service registry
API Gateway	8081	Centralna ulazna tačka, rutiranje, load balance
User Service	8086	Upravljanje korisnicima
Product Service	8082	Upravljanje proizvodima
Order Service	8083	Upravljanje narudžbinama
Review Service	8085	Upravljanje recenzijama
Notification Service	8084	Upravljanje notifikacijama
RabbitMQ	15672	Management konzola

3. Opis servisa

3.1. User Service

Servis zadužen za CRUD operacije nad korisnicima, koji radi na portu 8086. Ne zavisi ni od jednog drugog servisa (nema odlazne Feign pozive), pa nema ni Resilience4j konfiguraciju — jedini servis u sistemu bez eksternih poziva.

Metoda	Putanja	Opis
GET	/api/users	Lista svih korisnika
GET	/api/users/{id}	Prikaz jednog korisnika
POST	/api/users	Kreiranje korisnika
PUT	/api/users/{id}	Izmena korisnika
DELETE	/api/users/{id}	Brisanje korisnika

3.2. Product Service

Servis zadužen za CRUD operacije nad proizvodima, upravljanje zalihama i agregaciju podataka o proizvodu sa recenzijama iz Review servisa preko Feign klijenta. Radi na portu 8082.

Metoda	Putanja	Opis
GET	/api/products	Lista svih proizvoda
GET	/api/products/{id}	Prikaz jednog proizvoda
POST	/api/products	Kreiranje proizvoda
PUT	/api/products/{id}	Izmena proizvoda
DELETE	/api/products/{id}	Brisanje proizvoda
GET	/api/products/{id}/details	<i>Agregacioni endpoint</i> : proizvod + prosečna ocena + recenzije (Review servis)
POST	/api/products/reduce-stock/{id}	Umanjenje zalihe (Order servis poziva ovaj endpoint)
POST	/api/products/add-stock/{id}	Vraćanje zalihe (Order servis poziva ovaj endpoint)

3.3. Order Service

Servis zadužen za CRUD operacije nad narudžbinama, koji radi na portu 8083. Pri kreiranju narudžbine proverava postojanje korisnika i proizvoda preko Feign klijenata (User Service, Product Service), umanjuje zalihu proizvoda, čuva narudžbinu, i objavljuje OrderCreatedEvent na RabbitMQ radi asinhronog obaveštavanja, gde se na strani consumera(notification-service) kreira nova notifikacija. Takođe ima agregacioni endpoint koji kombinuje podatke iz tri servisa (Order, User, Product).

Metoda	Putanja	Opis
GET	/api/orders	Lista svih narudžbina
GET	/api/orders/{id}	Prikaz jedne narudžbine
POST	/api/orders	Kreiranje narudžbine (Feign + RabbitMQ event)
PUT	/api/orders/{id}	Izmena narudžbine (korekcija zalihe)

Metoda	Putanja	Opis
DELETE	/api/orders/{id}	Brisanje narudžbine
GET	/api/orders/{id}/details	Agregacija: narudžbina + korisnik + proizvod

3.4. Review Service

Servis zadužen za CRUD operacije nad recenzijama proizvoda, koji radi na portu 8085. Svaka recenzija je povezana i sa korisnikom i sa proizvodom, pa servis pri kreiranju/izmeni proverava postojanje oba preko Feign klijenata. Agregacioni endpoint kombinuje recenziju sa podacima o korisniku i proizvodu.

Metoda	Putanja	Opis
GET	/api/reviews	Lista svih recenzija
GET	/api/reviews/{id}	Prikaz jedne recenzije
POST	/api/reviews	Kreiranje recenzije
PUT	/api/reviews/{id}	Izmena recenzije
DELETE	/api/reviews/{id}	Brisanje recenzije
GET	/api/reviews/{id}/details	Agregacija: recenzija + korisnik + proizvod
GET	/api/reviews/product/{productId}	Sve recenzije za dati proizvod

3.5. Notification Service

Zadužen za CRUD operacije nad obaveštenjima korisnika. Notifikacije nastaju na dva načina: direktno preko REST API-ja, ili asinhrono, konzumiranjem OrderCreatedEvent poruka sa RabbitMQ-a nakon uspešno kreirane narudžbine.

Metoda	Putanja	Opis
GET	/api/notifications	Lista svih notifikacija
GET	/api/notifications/{id}	Prikaz jedne notifikacije
POST	/api/notifications	Kreiranje notifikacije
PUT	/api/notifications/{id}	Izmena notifikacije
DELETE	/api/notifications/{id}	Brisanje notifikacije
GET	/api/notifications/{id}/details	Agregacija: notifikacija + korisnik

Svaki servis poseduje springdoc-openapi zavisnost i Swagger UI na putanji /swagger-ui.html, gde je moguće pregledati i direktno testirati sve dostupne endpoint-e.

Svaki servis koristi odgovarajuće Request i Response DTO-ove.

4. Komunikacija između servisa

4.1. Sinhrona komunikacija — OpenFeign

Svi pozivi između servisa realizovani su preko OpenFeign klijenata koji koriste logičko ime servisa registrovano u Eureka (npr. `@FeignClient(name = "user-service")`), bez ijednog hardkodovanog URL-a. Stvarna adresa instance servisa razrešava se dinamički preko Eureka-e i Spring Cloud LoadBalancer-a.

Servis (klijent)	Poziva servis	Svrha poziva
Order Service	User Service	Provera postojanja korisnika pri kreiranju/izmeni narudžbine i dobavljanje korisnika za agregacioni endpoint
Order Service	Product Service	Provera postojanja proizvoda pri kreiranju/izmeni narudžbine, redukovanje zaliha i dobavljanje proizvoda za agregacioni endpoint
Review Service	User Service	Provera postojanja korisnika i dobavljanje korisnika za agregacioni endpoint

Servis (klijent)	Poziva servis	Svrha poziva
Review Service	Product Service	Provera postojanja proizvoda i dobavljanje proizvoda za agregacioni endpoint
Product Service	Review Service	Dobavljanje recenzija za agregacioni endpoint
Notification Service	User Service	Provera postojanja korisnika i dobavljanje korisnika za agregacioni endpoint

4.1.1 Otpornost na greške — Resilience4j

Svaki eksterni (Feign) poziv zaštićen je kombinacijom Circuit Breaker-a i Retry mehanizma, sa definisanom fallback metodom koja u slučaju nedostupnosti zavisnog servisa baca `ServiceUnavailableException` (mapiran na HTTP 503). Legitimno nepostojanje resursa (HTTP 404 od zavisnog servisa) hvata se unutar metode i ne prosleđuje se Circuit Breaker-u, kako lažni negativni (nepostojeći ID) ne bi uticali na statistiku otvaranja kola.

Parametar	Vrednost
sliding-window-size	5
failure-rate-threshold	50%
minimum-number-of-calls	5
wait-duration-in-open-state	10s
permitted-number-of-calls-in-half-open-state	3
Retry — max-attempts	3
Retry — wait-duration	1s

Poseban slučaj je poziv za umanjeње zalihe u Order Service-u: ako Product Service vrati HTTP 409 (nedovoljna zaliha), ta greška se tretira kao poslovna (`InsufficientStockException`, HTTP 409), a ne kao infrastrukturni kvar servisa, i eksplicitno je izuzeta iz Circuit Breaker/Retry statistike (`ignoreExceptions`) da ne bi lažno otvarala kolo.

4.3. Asinhrona komunikacija — RabbitMQ

Nakon uspešnog kreiranja narudžbine, Order Service (producer) objavljuje `OrderCreatedEvent` poruku na Topic Exchange (`order.exchange`, routing key `order.created`). Notification Service (consumer) osluškuje red `notification.order.created.queue` preko `@RabbitListener` anotacije i, po prijemu poruke, kreira novu notifikaciju za korisnika. Poruke se serijalizuju u JSON format (`Jackson2JsonMessageConverter`) radi čitljivosti i nezavisnosti od jezika/platforme.

Idempotentnost na strani cosumera obezbeđena je preko jedinstvenog identifikatora događaja (`eventId`, UUID) i tabele `processed_event`: pre obrade poruke proverava se da li je `eventId` već obrađen; ako jeste, poruka se ignoriše, čime se sprečava duplo kreiranje notifikacije u slučaju da RabbitMQ (at-least-once isporuka) istu poruku isporuči više puta.

5. API Gateway i Service Discovery

API Gateway predstavlja centralnu ulaznu tačku u sistem za spoljne klijente. Rute su definisane u `application.yaml` sa `lb://` prefiksom, što znači da se svaki zahtev prosleđuje kroz load balancer koji dinamički bira dostupnu instancu servisa preko Eureka registra.

Putanja (predicate)	Ciljni servis
<code>/api/users/**</code>	<code>lb://user-service</code>
<code>/api/products/**</code>	<code>lb://product-service</code>
<code>/api/orders/**</code>	<code>lb://order-service</code>
<code>/api/reviews/**</code>	<code>lb://review-service</code>
<code>/api/notifications/**</code>	<code>lb://notification-service</code>

Eureka Discovery Server radi kao samostalan registar (`register-with-eureka: false`, `fetch-registry: false`) i ne registruje sebe. Svi ostali servisi, uključujući Gateway, registruju se kao Eureka klijenti sa `prefer-ip-address: true`, čime se osigurava pouzdana registracija i unutar Docker mreže.

6. Rukovanje greškama

Svaki poslovni servis ima sopstveni `GlobalExceptionHandler` (`@RestControllerAdvice`) koji na konzistentan način mapira izuzetke na HTTP statuse i jedinstven JSON format greške (`status`, `error`, `message`, `timestamp`).

Izuzetak	HTTP status	Značenje
<code>NotFoundException</code>	404 Not Found	Traženi resurs ne postoji
<code>InsufficientStockException</code>	409 Conflict	Nedovoljna zaliha proizvoda
<code>ServiceUnavailableException</code>	503 Service Unavailable	Zavisan servis trenutno nedostupan (fallback)
<code>MethodArgumentNotValidException</code>	400 Bad Request	Neuspešan Bean Validation
<code>Exception</code> (generički)	500 Internal Server Error	Sve ostale greške

7. Kontejnerizacija — Docker i Docker Compose

Svaki od sedam servisa ima sopstveni Dockerfile organizovan kao multi-stage build: prva faza (`maven:3.9.9-eclipse-temurin-17`) kompajlira izvorni kod i pravi izvršni `.jar`, a druga, znatno manja faza (`eclipse-temurin:17-jre-alpine`) sadrži samo taj `.jar` i JRE potreban za pokretanje, bez Maven-a i izvornog koda.

Ceo sistem se podiže jednom komandom preko `docker-compose.yml`, koji definiše svih sedam kontejnera, `healthcheck` provere (Eureka i RabbitMQ) i `depends_on` uslove (`service_healthy`) koji garantuju da se servisi pokreću tek kada su njihove zavisnosti stvarno spremne, a ne samo pokrenute.

Eureka URL i RabbitMQ host se u Docker okruženju prosleđuju preko environment varijable (`EUREKA_CLIENT_SERVICEURL_DEFAULTZONE`, `SPRING_RABBITMQ_HOST`) koje prepisuju podrazumevane vrednosti iz `application.yaml`.